

Spring Security – Complete Notes (Spring Boot 3.x & Spring Security 6.x)

Scope (what you'll learn)

Authentication vs Authorization • Security filter chain internals • Thread-local security context
• Form Login (stateful) • In-Memory users • DB/JPA-backed users • Method security • CSRF/
CORS/Headers • Sessions/Remember-me • JWT (stateless) with Spring's Resource Server •
Testing & troubleshooting.

1) High-level mental model

Request → `FilterChainProxy` → Security filters → Authentication → `SecurityContext` →
Authorization checks → Controller

- **Authentication** answers *who are you?* (establishes a `Principal`).
- **Authorization** answers *can you do this?* (`GrantedAuthority` / roles checked against rules).
- **SecurityContext** (per request) holds the `Authentication` object and lives in a `ThreadLocal` via `SecurityContextHolder`.
- **Filters**: ordered Servlet filters that implement the whole security flow. You don't usually touch most of them; you configure them through the DSL in your `SecurityFilterChain` bean.

Core types

Concept	Key class/bean	What it does
Current user	<code>Authentication</code> (e.g., <code>UsernamePasswordAuthenticationToken</code>)	Holds principal name, authorities, and <code>authenticated=true/false</code> .
Store & access	<code>SecurityContext</code> + <code>SecurityContextHolder</code>	Thread-local access to the current <code>Authentication</code> .
Auth engine	<code>AuthenticationManager</code> (default <code>ProviderManager</code>)	Delegates to one or more <code>AuthenticationProvider</code> s.
Local provider	<code>DaoAuthenticationProvider</code>	Loads user via <code>UserDetailsService</code> , checks password via <code>PasswordEncoder</code> .
Users API	<code>UserDetailsService</code>	Lookup user by username; returns a <code>UserDetails</code> (username, password, authorities, flags).

Concept	Key class/bean	What it does
Passwords	PasswordEncoder (e.g., BCryptPasswordEncoder)	One-way hashing; never store raw passwords.
URL auth	AuthorizationFilter + authorization rules	Enforces authorizeHttpRequests rules.
Method auth	@PreAuthorize, @Secured, @PostAuthorize	Enforced after URL auth, at method boundary.

Filter ordering (simplified)

CorsFilter → CsrfFilter → LogoutFilter → UsernamePasswordAuthenticationFilter (form login) / BearerTokenAuthenticationFilter (JWT) / BasicAuthenticationFilter → AnonymousAuthenticationFilter → ExceptionTranslationFilter → AuthorizationFilter / FilterSecurityInterceptor .

ExceptionTranslationFilter converts AuthenticationException → 401 via AuthenticationEntryPoint, and AccessDeniedException → 403 via AccessDeniedHandler .

2) Authentication internals (deep dive)

1. **Credentials extraction** from request by a filter (e.g., form login reads username & password, JWT reads Authorization: Bearer ...).
2. Filter builds an **unauthenticated** Authentication token (e.g., UsernamePasswordAuthenticationToken) and calls AuthenticationManager.authenticate(..).
3. ProviderManager iterates its configured AuthenticationProvider s. The matching provider (e.g., DaoAuthenticationProvider) will:
4. loadUserByUsername(..) via your UserDetailsService .
5. Verify password with your PasswordEncoder.matches(raw, encoded) .
6. On success, return an **authenticated** Authentication containing principal + GrantedAuthority s.
7. The filter stores it into SecurityContextHolder.getContext().setAuthentication(..) and delegates down the chain.
8. Later, the **Authorization phase** consults this authentication for decisions.

SecurityContext persistence - **Form login/stateful**: context is saved to **HTTP session** via SecurityContextRepository (HttpSessionSecurityContextRepository), so subsequent requests are already authenticated. - **JWT/stateless**: context is **rebuilt every request** from the token. SessionCreationPolicy.STATELESS and CSRF disabled.

Anonymous user - When no authentication exists, AnonymousAuthenticationFilter populates an AnonymousAuthenticationToken (authority ROLE_ANONYMOUS) so authorization can still evaluate expressions.

3) Authorization internals

- **URL authorization** (request level): configured via `authorizeHttpRequests`. Under the hood, Spring Security 6 uses **AuthorizationManager** to evaluate matchers and expressions like `hasRole('ADMIN')`, `hasAuthority('SCOPE_read')`, etc.
- **Method security** (service layer protection): enable with `@EnableMethodSecurity`. Common annotations:
 - `@PreAuthorize("hasRole('ADMIN')")`
 - `@PostAuthorize("returnObject.owner == authentication.name")`
 - `@PreAuthorize("hasAuthority('order:read')")`

Roles vs Authorities - **Role** is a convention: `ROLE_ADMIN` is an authority whose name starts with `ROLE_`.
- DSL helpers: - `hasRole('ADMIN')` → checks for authority `ROLE_ADMIN`. - `hasAuthority('ROLE_ADMIN')` → explicit authority check.

Hierarchy (optional) - You can model **role hierarchy** (e.g., `ROLE_ADMIN > ROLE_STAFF > ROLE_USER`) so higher roles inherit lower authorities.

4) Security configuration templates

Spring Boot 3/Security 6 use `SecurityFilterChain` **beans** instead of the old `WebSecurityConfigurerAdapter`.

4.1 Minimal secure defaults (stateful, browser app)

```
@Configuration
@EnableMethodSecurity
public class SecurityConfig {
    @Bean
    SecurityFilterChain web(HttpSecurity http) throws Exception {
        http
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/public/**", "/css/**", "/js/**", "/images/
**").permitAll()
                .anyRequest().authenticated()
            )
            .formLogin(form -> form
                .loginPage("/login").permitAll()           // custom login page endpoint
                .defaultSuccessUrl("/home", true)
                .failureUrl("/login?error=true")
            )
            .logout(logout -> logout
                .logoutUrl("/logout")
            )
    }
}
```

```

        .logoutSuccessUrl("/login?logout=true")
    )
    .csrf(csrf -> csrf
        // For browser forms, keep CSRF enabled. For SPAs, consider:
        .csrfTokenRepository(CookieCsrfTokenRepository.withHttpOnlyFalse())
    )
    .headers(h -> h.contentSecurityPolicy(csp ->
csp.policyDirectives("default-src 'self'")));
    return http.build();
}

@Bean PasswordEncoder passwordEncoder() { return new
BCryptPasswordEncoder(); }
}

```

4.2 In-Memory users (quick POC)

```

@Bean
UserDetailsService users(PasswordEncoder encoder) {
    UserDetails admin = User.withUsername("admin")
        .password(encoder.encode("admin123"))
        .roles("ADMIN")    // translates to authority ROLE_ADMIN
        .build();
    UserDetails user = User.withUsername("user")
        .password(encoder.encode("user123"))
        .roles("USER")
        .build();
    return new InMemoryUserDetailsManager(admin, user);
}

```

Tip: Never use `{noop}` in real apps. Prefer BCrypt (`BCryptPasswordEncoder`).

5) Form Login – end-to-end (stateful)

Request flow

1. User opens `/login` (your MVC page).
2. Submits POST to `/login` with parameters `username` and `password` (configurable).
3. `UsernamePasswordAuthenticationFilter` triggers authentication.
4. On success, the context is saved into the session and browser receives a **session cookie (JSESSIONID)**.

Customizations you'll often need

```
.formLogin(form -> form
  .loginPage("/login")
  .loginProcessingUrl("/do-login")      // if your form posts to /do-login
  .usernameParameter("email")          // if you use email instead of username
  .passwordParameter("pwd")
  .successHandler((req,res,auth) -> res.sendRedirect("/dashboard"))
  .failureHandler((req,res,ex) -> res.sendRedirect("/login?error"))
)
```

Remember-me (optional)

- **Cookie-based:** short-lived hash cookie.
- **Persistent/JDBC:** `PersistentTokenRepository` stores tokens in DB.

```
.rememberMe(r -> r
  .tokenValiditySeconds(1209600) // 14 days
  .userDetailsService(users)
)
```

Session management & fixation

```
.sessionManagement(sm -> sm
  .sessionFixation(sf -> sf.migrateSession()) // default: prevent session
  fixation
  .maximumSessions(1)                        // single-session per user
)
```

6) Managing users in a Database (JPA)

Schema (simple, normalized)

```
users(id PK, username unique, password, enabled, account_non_locked,
account_non_expired, credentials_non_expired)
roles(id PK, name unique)          -- e.g., ROLE_ADMIN, ROLE_USER
user_roles(user_id FK users, role_id FK roles) (unique user_id, role_id)
```

JPA entities

```
@Entity
@Table(name = "users")
public class AppUser {
    @Id @GeneratedValue private Long id;
    @Column(unique = true, nullable = false) private String username;
    @Column(nullable = false) private String password;
    private boolean enabled = true;
    private boolean accountNonLocked = true;
    private boolean accountNonExpired = true;
    private boolean credentialsNonExpired = true;

    @ManyToMany(fetch = FetchType.EAGER)
    @JoinTable(name = "user_roles",
        joinColumns = @JoinColumn(name = "user_id"),
        inverseJoinColumns = @JoinColumn(name = "role_id"))
    private Set<Role> roles = new HashSet<>();
}

@Entity
@Table(name = "roles")
public class Role {
    @Id @GeneratedValue private Long id;
    @Column(unique = true, nullable = false) private String name; // e.g.,
    ROLE_ADMIN
}
```

Repositories

```
public interface UserRepository extends JpaRepository<AppUser, Long> {
    Optional<AppUser> findByUsername(String username);
}
```

Map to `UserDetails`

```
@Service
public class JpaUserDetailsService implements UserDetailsService {
    private final UserRepository users;
    public JpaUserDetailsService(UserRepository users) { this.users = users; }

    @Override
    public UserDetails loadUserByUsername(String username) throws
    UsernameNotFoundException {
```

```

AppUser u = users.findByUsername(username)
                    .orElseThrow(() -> new UsernameNotFoundException("User not
found"));
List<GrantedAuthority> auth = u.getRoles().stream()
    .map(r -> new SimpleGrantedAuthority(r.getName()))
    .toList();
return new org.springframework.security.core.userdetails.User(
    u.getUsername(), u.getPassword(), u.isEnabled(),
    u.isAccountNonExpired(), u.isCredentialsNonExpired(),
    u.isAccountNonLocked(), auth);
}
}

```

Hook into auth

```

@Configuration
@EnableMethodSecurity
public class SecurityConfig {
    private final JpaUserDetailsService uds;
    public SecurityConfig(JpaUserDetailsService uds) { this.uds = uds; }

    @Bean SecurityFilterChain web(HttpSecurity http) throws Exception {
        http
            .authorizeHttpRequests(a -> a
                .requestMatchers("/auth/**", "/public/**").permitAll()
                .anyRequest().authenticated())
            .formLogin(withDefaults());
        return http.build();
    }

    @Bean PasswordEncoder passwordEncoder() { return new
BCryptPasswordEncoder(); }

    // For programmatic authentication (e.g., custom login controllers if needed)
    @Bean AuthenticationManager authenticationManager(AuthenticationConfiguration
cfg) throws Exception {
        return cfg.getAuthenticationManager();
    }
}

```

Creating users safely

```

@Service
public class UserAdminService {
    private final UserRepository repo; private final PasswordEncoder encoder;

```

```

    public UserAdminService(UserRepository r, PasswordEncoder e){ this.repo=r;
    this.encoder=e; }

    public AppUser create(String username, String rawPassword, Set<Role> roles){
        AppUser u = new AppUser();
        u.setUsername(username);
        u.setPassword(encoder.encode(rawPassword));
        u.setRoles(roles);
        return repo.save(u);
    }
}

```

Tip: Use **DelegatingPasswordEncoder** prefixes (`{bcrypt}`, `{argon2}`) for smooth algorithm upgrades.

7) Method security examples

```

@Service
public class OrderService {
    @PreAuthorize("hasRole('ADMIN') or (#customerId == authentication.name)")
    public OrderDto getOrder(String customerId, String orderId) { ... }

    @PreAuthorize("hasAuthority('order:write')")
    public void updateOrder(UpdateOrder cmd) { ... }
}

```

Enable with:

```

@Configuration
@EnableMethodSecurity(prePostEnabled = true, securedEnabled = true)
class MethodSecConfig {}

```

Custom permission evaluator (advanced): implement `PermissionEvaluator` and reference it from SpEL with `hasPermission(target, 'edit')`.

8) CSRF, CORS, headers

- **CSRF** protects browser sessions against cross-site request forgery.
- **Enable** for stateful browser apps (default).
- **Disable** for stateless APIs (no session/cookie) or use token-based auth.

- **CORS** controls cross-origin requests for SPAs/mobile apps; configure allowed origins/methods/headers.
- **Security headers:** Spring sets many by default (HSTS, X-Content-Type-Options, X-Frame-Options). Adjust via `headers(...)`.

```
http
.cors(cors -> cors.configurationSource(req -> {
    CorsConfiguration cfg = new CorsConfiguration();
    cfg.setAllowedOrigins(List.of("https://your-spa.example"));
    cfg.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE"));
    cfg.setAllowedHeaders(List.of("Authorization", "Content-Type"));
    return cfg;
})))
.csrf(csrf -> csrf.disable()); // only for stateless APIs
```

9) JWT Authentication (Stateless)

Modern & recommended path: **use Spring's OAuth2 Resource Server**. It validates Bearer tokens for you. You can mint tokens yourself using Spring's `JwtEncoder` (HS256 or RSA) or integrate with an external IdP like Keycloak/Okta.

9.1 Maven dependencies

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-oauth2-resource-server</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.security</groupId>
  <artifactId>spring-security-oauth2-jose</artifactId>
</dependency>
```

9.2 Security config (resource server + stateless)

```
@Configuration
@EnableMethodSecurity
public class SecurityConfig {
    @Bean SecurityFilterChain api(HttpSecurity http) throws Exception {
        http
```

```

        .csrf(csrf -> csrf.disable())
        .sessionManagement(sm ->
sm.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/api/auth/**", "/public/**").permitAll()
            .anyRequest().authenticated())
        .oauth2ResourceServer(oauth2 -> oauth2.jwt(withDefaults()));
    return http.build();
}

@Bean PasswordEncoder passwordEncoder() { return new
BCryptPasswordEncoder(); }
@Bean AuthenticationManager authenticationManager(AuthenticationConfiguration
cfg) throws Exception { return cfg.getAuthenticationManager(); }
}

```

9.3 Signing & verifying tokens (HS256 example)

```

@Configuration
class JwtConfig {
    private static final String SECRET = "change-me-256-bit-secret"; // use env
var & >= 32 bytes

    @Bean JwtEncoder jwtEncoder() {
        SecretKey key = new SecretKeySpec(SECRET.getBytes(StandardCharsets.UTF_8),
"HmacSHA256");
        return new NimbusJwtEncoder(new ImmutableSecret<>(key));
    }
    @Bean JwtDecoder jwtDecoder() {
        SecretKey key = new SecretKeySpec(SECRET.getBytes(StandardCharsets.UTF_8),
"HmacSHA256");
        return NimbusJwtDecoder.withSecretKey(key).build();
    }
}

```

9.4 Login endpoint that mints JWT

```

@RestController
@RequestMapping("/api/auth")
public class AuthController {
    private final AuthenticationManager authManager; private final JwtEncoder
encoder;
    public AuthController(AuthenticationManager am, JwtEncoder enc){
this.authManager = am; this.encoder = enc; }
}

```

```

record LoginRequest(String username, String password) {}
record TokenResponse(String accessToken, long expiresIn) {}

@PostMapping("/login")
public TokenResponse login(@RequestBody LoginRequest req) {
    Authentication auth = authManager.authenticate(
        new UsernamePasswordAuthenticationToken(req.username(),
req.password()));

    Instant now = Instant.now();
    String scope = auth.getAuthorities().stream()
        .map(GrantedAuthority::getAuthority).collect(Collectors.joining(" "));

    JwtClaimsSet claims = JwtClaimsSet.builder()
        .issuer("self")
        .issuedAt(now)
        .expiresAt(now.plusSeconds(3600))
        .subject(auth.getName())
        .claim("scope", scope)
        .build();

    String token =
encoder.encode(JwtEncoderParameters.from(claims)).getTokenValue();
    return new TokenResponse(token, 3600);
}
}

```

9.5 Use the token

- Client sends: `Authorization: Bearer <token>`.
- Resource server decodes & validates. If valid, `JwtAuthenticationToken` is placed in the `SecurityContext` with `GrantedAuthority`s from scopes/roles you map (configure a converter if needed).

Authority mapping from `scope` claim

```

@Bean
JwtAuthenticationConverter jwtAuthenticationConverter() {
    JwtGrantedAuthoritiesConverter scopes = new JwtGrantedAuthoritiesConverter();
    scopes.setAuthorityPrefix("SCOPE_"); // default
    scopes.setAuthoritiesClaimName("scope");

    JwtAuthenticationConverter conv = new JwtAuthenticationConverter();
    conv.setJwtGrantedAuthoritiesConverter(scopes);
    return conv;
}

```

```
@Bean SecurityFilterChain api(HttpSecurity http, JwtAuthenticationConverter
conv) throws Exception {
    http.oauth2ResourceServer(o -> o.jwt(j ->
j.jwtAuthenticationConverter(conv)));
    return http.build();
}
```

Why resource server? You avoid writing custom JWT filters, clock skew issues, error handling, etc. Spring handles validation, exception translation, and places a proper `JwtAuthenticationToken` into the context.

10) Common patterns & pitfalls

- **CSRF disabled on APIs** but don't disable it for MVC form apps.
- **Password encoder mismatch** → There is no PasswordEncoder mapped for the id "null". Fix by storing passwords with an id (e.g., `{bcrypt}hash`). If you use `BCryptPasswordEncoder`, Spring automatically prefixes `{bcrypt}`.
- **Missing** `AuthenticationManager` bean when writing custom login controller? Inject via `AuthenticationConfiguration`.
- **Permit static assets** or your CSS/JS on login page won't load.
- **Role prefix confusion:** `hasRole('ADMIN')` expects authority `ROLE_ADMIN`. Use `hasAuthority('ROLE_ADMIN')` or configure a different prefix.
- **Order of matchers matters:** more specific `requestMatchers` first.
- **Stateless JWT + Form login together?** Keep them separate (e.g., different `SecurityFilterChain`s by `securityMatcher`), or move entirely to token auth for APIs.

11) Testing security

Unit/service level

- Use `@WithMockUser(roles = "ADMIN")` on tests of methods secured with `@PreAuthorize`.

MVC/integration

```
mockMvc.perform(post("/login").param("username", "user").param("password", "pw"))
    .andExpect(authenticated());

mockMvc.perform(get("/api/orders").with(jwt().jwt(jwt ->
jwt.claim("scope", "order:read"))))
    .andExpect(status().isOk());
```

12) End-to-end recipes

A) Simple MVC app with form login

1. Add `spring-boot-starter-security`, `spring-boot-starter-thymeleaf`.
2. Configure `SecurityFilterChain` with `.formLogin()` and keep CSRF enabled.
3. Provide `UserDetailsService` (in-memory or JPA) + `PasswordEncoder`.
4. Create `/login` page and controllers.
5. Protect `/admin/**` with `hasRole('ADMIN')` and others as needed.

B) REST API with JWT

1. Add `security`, `oauth2-resource-server`, `oauth2-jose` dependencies.
2. Define `JwtEncoder` + `JwtDecoder` (HS256 or RSA) and a login controller that mints tokens.
3. Configure `SecurityFilterChain` with `STATELESS`, `.oauth2ResourceServer(jwt())`, CSRF disabled.
4. Annotate endpoints with `@PreAuthorize` or configure granular `authorizeHttpRequests` rules.

13) Operational checklists

- **Secrets:** store JWT secrets/keys in env vars or Secret Manager; rotate periodically.
- **Password policy:** min length, complexity, breach check (HIBP), rate limiting on login.
- **Account flags:** lock after N failed attempts; unlock strategy.
- **Logging/Auditing:** success/failure logins, token issuance, privileged actions.
- **HTTPS everywhere:** set `secure` cookies; HSTS header on.
- **Refresh tokens:** if you mint short-lived access tokens, consider a refresh token flow.

14) Quick reference (cheatsheet)

- Default login processing URL: `/login` (POST).
- Default param names: `username`, `password`.
- Permit rules DSL:

```
.authorizeHttpRequests(a -> a
    .requestMatchers(HttpMethod.GET, "/health").permitAll()
    .requestMatchers("/admin/**").hasRole("ADMIN")
    .requestMatchers(HttpMethod.POST, "/orders/
    **").hasAuthority("order:write")
    .anyRequest().authenticated())
```

- Get the current user in code: `Authentication a = SecurityContextHolder.getContext().getAuthentication();` or in MVC `@AuthenticationPrincipal`.

Appendix A: Multiple `SecurityFilterChain`s

- You can define **multiple chains** and apply them selectively:

```
@Bean SecurityFilterChain api(HttpSecurity http) throws Exception {
    http.securityMatcher("/api/**")
        .sessionManagement(sm ->
            sm.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .oauth2ResourceServer(o -> o.jwt())
        .authorizeHttpRequests(a -> a.anyRequest().authenticated());
    return http.build();
}

@Bean SecurityFilterChain web(HttpSecurity http) throws Exception {
    http.securityMatcher("/**")
        .formLogin(withDefaults())
        .authorizeHttpRequests(a -> a.anyRequest().authenticated());
    return http.build();
}
```

Appendix B: Password upgrade strategy

- If you migrate from plain hashes, use `DelegatingPasswordEncoder` and **re-encode** on next login:

```
String newHash = passwordEncoder.encode(raw);
user.setPassword(newHash); // save after successful login
```

You now have both the internals and the practical recipes to build:

1) a classic session-based MVC app, and 2) a modern stateless JWT API—both with method-level authorization and strong password handling.